

LIGERITO



Diego Fernández López

Motivación.	2
Problema que soluciona.....	2
Posibles usuarios.....	2
Escalabilidad.....	3
Requisitos funcionales.....	3
Requisitos no funcionales.	5
Diagrama Entidad-Relación	6
Tablas y atributos.....	6
Diagramas Spring	9
Arquitectura del backend	10
Base de datos.....	11
Diagrama de clases completo.....	12
Mapa navegación entre pantallas.....	13
Estructura del frontend	13
Tecnologías utilizadas.	19
Conclusiones	19

Motivación.

Como apasionado de la montaña, considero que una de las tareas críticas previas a cualquier actividad es la preparación del equipo. Mientras que para una salida de un día la planificación puede ser sencilla, en actividades de varias jornadas o en disciplinas de ultradistancia, la gestión del peso y del material se vuelve un factor vital para garantizar tanto el éxito como la seguridad de la aventura.

Problema que soluciona.

Ligerito permite a los usuarios gestionar su inventario de artículos deportivos y configurar mochilas o equipajes específicos para cada actividad. La aplicación ofrece una visión detallada del peso total y un desglose claro por categorías, permitiendo optimizar la carga de forma visual y altamente eficiente. Además, fomenta el aspecto social y de comunidad al permitir a los usuarios compartir sus configuraciones de material con otros deportistas.

Posibles usuarios.

El público objetivo abarca desde entusiastas de la ultradistancia hasta perfiles más cotidianos. Principalmente, la aplicación resulta indispensable para usuarios que practican senderismo de gran recorrido o viajes en bicicleta (bikepacking), ya sea con un enfoque competitivo, deportivo o puramente recreativo.

Aplicación en Alta Montaña y Senderismo:

- **Anillo de Picos de Europa:** Travesías de hasta 110 km y más de 9.000 m de desnivel positivo, donde cada gramo cuenta en las trepadas.
- **GR-11 (La Transpirenaica):** Ruta que une el Cantábrico (Cabo Higuer) con el Mediterráneo (Cap de Creus) a través de los Pirineos en unas 40 etapas.
- **GR-7 (Sendero Europeo E-4):** El sendero balizado más largo de España, cruzando la península desde Andorra hasta Tarifa (Cádiz).

Aplicación en Ciclismo y Bikepacking:

- **Transpyrenees:** Prueba de autosuficiencia de unos 1.000 kilómetros y más de 25.000 metros de desnivel cruzando los Pirineos, donde el equilibrio y peso de las bolsas es crucial.
- **Montañas Vacías:** Recorrido circular *off-road* de 680 km y 13.000 metros de desnivel a través de la "Laponia española" (Serranía de Cuenca, Montes Universales y Gúdar-Javalambre).

Escalabilidad.

Durante las fases de presentación y pruebas del proyecto, el *feedback* recibido por parte de profesores como José Luis y Germán reveló un valor añadido fundamental: Ligerito no solo es una herramienta para el rendimiento extremo. Su sistema de inventario es igualmente útil como "lista de verificación" para evitar olvidos en el día a día. Resulta una ayuda perfecta para organizar la bolsa de la piscina de los niños, preparar el equipo para una sesión de windsurf en verano o planificar cualquier escapada de fin de semana, garantizando que la actividad se desarrolle sin imprevistos logísticos.

Requisitos funcionales.

Los requisitos se han reestructurado para reflejar un sistema de inventario dinámico y persistente, priorizados según su impacto en la experiencia del usuario.

1. Gestión de Usuarios y Acceso (Prioridad Crítica)

- **RF-01. Registro de usuario:** El sistema permitirá crear cuentas únicas vinculadas a un correo electrónico.
- **RF-02. Autenticación (Login):** Acceso seguro al ecosistema personal (mochilas, armario y categorías).
- **RF-03. Cierre de sesión (Logout):** Finalización de sesión para garantizar la privacidad de los datos.

2. Gestión de Mochilas y Control de Peso (Prioridad Alta)

- **RF-04. Creación de listas:** El usuario podrá generar múltiples mochilas con nombres personalizados.
- **RF-05. Cálculo de peso total (CRÍTICO):** El sistema automatiza el cálculo del peso total en tiempo real:

$$Peso_{Total} = \sum_{i=1}^n (Peso_{item_i} \times Cantidad_i)$$

Desglose de la fórmula

La ecuación se lee como: "El **Peso Total** es la suma de los pesos individuales multiplicados por su cantidad".

- **$Peso_{Total}$:** Es el resultado final (el peso de la mochila completa o de una categoría específica).

- $\sum_{i=1}^n$: Este es el símbolo de **Sumatorio**. Indica que vamos a sumar una lista de elementos, empezando por el primero ($i=1$) hasta llegar al último (n). En tu caso, n es el número total de artículos diferentes que has metido en la mochila.
 - **$Peso_{item_i}$** : Es el peso unitario del artículo número i . Por ejemplo, si el artículo 1 es una "Barrita energética", su peso unitario serían 50g.
 - **$Cantidad_i$** : Es el número de unidades que llevas de ese artículo específico. Siguiendo el ejemplo, si llevas 5 barritas, la cantidad es 5.
 - **$(Peso_{item_i} \times Cantidad_i)$** : Es el peso parcial por cada tipo de artículo.
- **RF-06. Desglose de peso por categorías:** Visualización del peso acumulado por cada categoría creada por el usuario.
 - **RF-07. Gestión de visibilidad de la mochila:** El usuario podrá cambiar el estado de una mochila entre pública y privada. Cuando una mochila se marque como pública, aparecerá en la sección de exploración para que otros usuarios puedan visitarla y consultarla en modo lectura.
 - **RF-08. Eliminación definitiva:** El usuario podrá borrar cualquier mochila, incluida la última, previa confirmación de seguridad. Si no existen mochilas, el sistema permanecerá en estado vacío hasta la creación de una nueva.

3. Gestión de Inventario y Personalización (Prioridad Media)

- **RF-09. Inventario Maestro (Armario):** El sistema mantendrá un inventario personal por usuario donde se registrarán todos los objetos reutilizables creados, permitiendo su uso en diferentes mochilas sin duplicar datos.
- **RF-10. Categorías personalizadas por mochila:** El sistema permitirá al usuario crear, nombrar y gestionar categorías propias dentro de cada mochila para organizar el equipamiento y calcular el reparto de peso por secciones.
- **RF-11. Alta de ítems reutilizables con metadatos:** El usuario podrá crear objetos desde una mochila, indicando nombre, peso en gramos, descripción técnica opcional y enlace opcional. Una vez creados, estos pasarán al armario personal para poder reutilizarse en otras mochilas sin duplicar datos.
- **RF-12. Control dinámico de unidades:** El usuario podrá incrementar o decrementar la cantidad de un objeto dentro de una mochila, actualizándose automáticamente el contenido de la lista y el peso total asociado.

- **RF-13. Sincronización de datos globales del objeto:** El sistema permitirá modificar el peso, la descripción y, en su caso, otros metadatos globales de un objeto reutilizable desde cualquier mochila donde aparezca, propagando el cambio al armario general y al resto de mochilas que reutilicen dicho objeto.

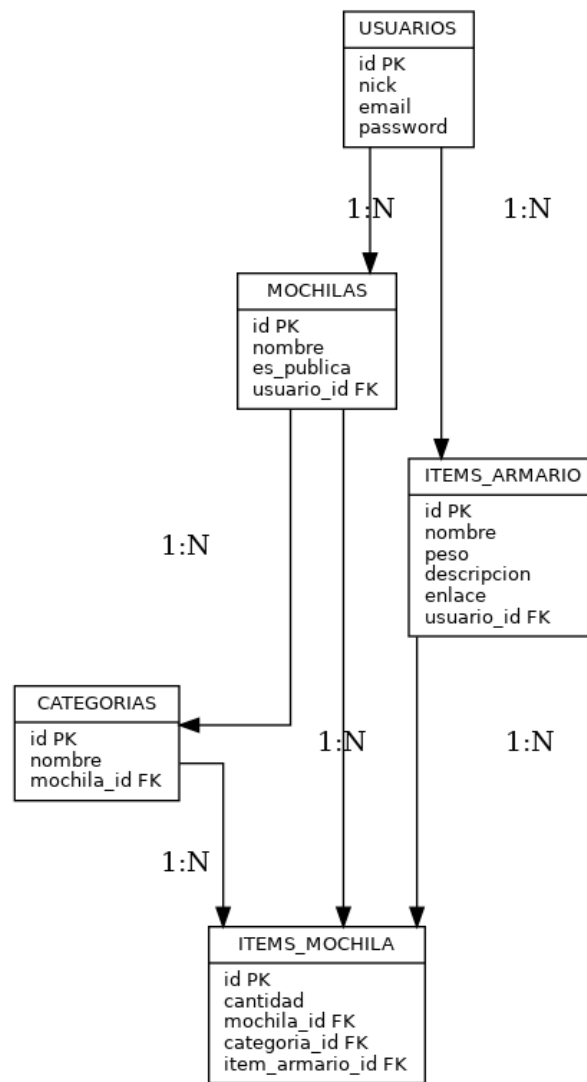
4. Visualización y Comunidad (Prioridad Baja)

- **RF-14. Exploración de comunidad:** Acceso en modo lectura a las mochilas marcadas como públicas por otros usuarios para servir de inspiración o guía.

Requisitos no funcionales.

- **RNF-01. Usabilidad:** La aplicación deberá ofrecer una interfaz clara, sencilla e intuitiva, de forma que el usuario pueda registrarse, iniciar sesión, gestionar mochilas y consultar su armario sin necesidad de formación previa.
- **RNF-02. Persistencia de datos:** El sistema deberá almacenar de forma persistente la información de usuarios, mochilas, categorías y objetos, evitando su pérdida al cerrar la aplicación o reiniciar el sistema.
- **RNF-03. Consistencia de datos:** El sistema deberá mantener la coherencia entre el armario general y las mochilas del usuario, garantizando que los cambios en los datos globales de un objeto se reflejen correctamente en todos los lugares donde se reutilice.
- **RNF-04. Seguridad básica:** El acceso a la información personal deberá estar restringido a usuarios autenticados, impidiendo el acceso o modificación de datos ajenos.
- **RNF-05. Mantenibilidad:** El proyecto deberá estar organizado de forma modular y estructurada, separando responsabilidades entre interfaz, lógica de negocio y persistencia, para facilitar su comprensión, corrección y ampliación futura.
- **RNF-06. Validación de entradas:** El sistema deberá validar los datos introducidos por el usuario en formularios y operaciones, comprobando que campos como email, contraseña, nombre, peso o cantidad tengan un formato y valor correctos antes de ser procesados.

Diagrama Entidad-Relación



Tablas y atributos.

Usuarios		
Columna	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
nick	VARCHAR	UNIQUE, NOT NULL
email	VARCHAR	UNIQUE, NOT NULL
password	VARCHAR	NOT NULL

Mochilas		
Columna	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
nombre	VARCHAR	NOT NULL
es_publica	BOOLEAN	NOT NULL
usuario_id	BIGINT	FK → usuarios.id

Categorías		
Columna	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
nombre	VARCHAR	NOT NULL
mochila_id	BIGINT	FK → mochilas.id

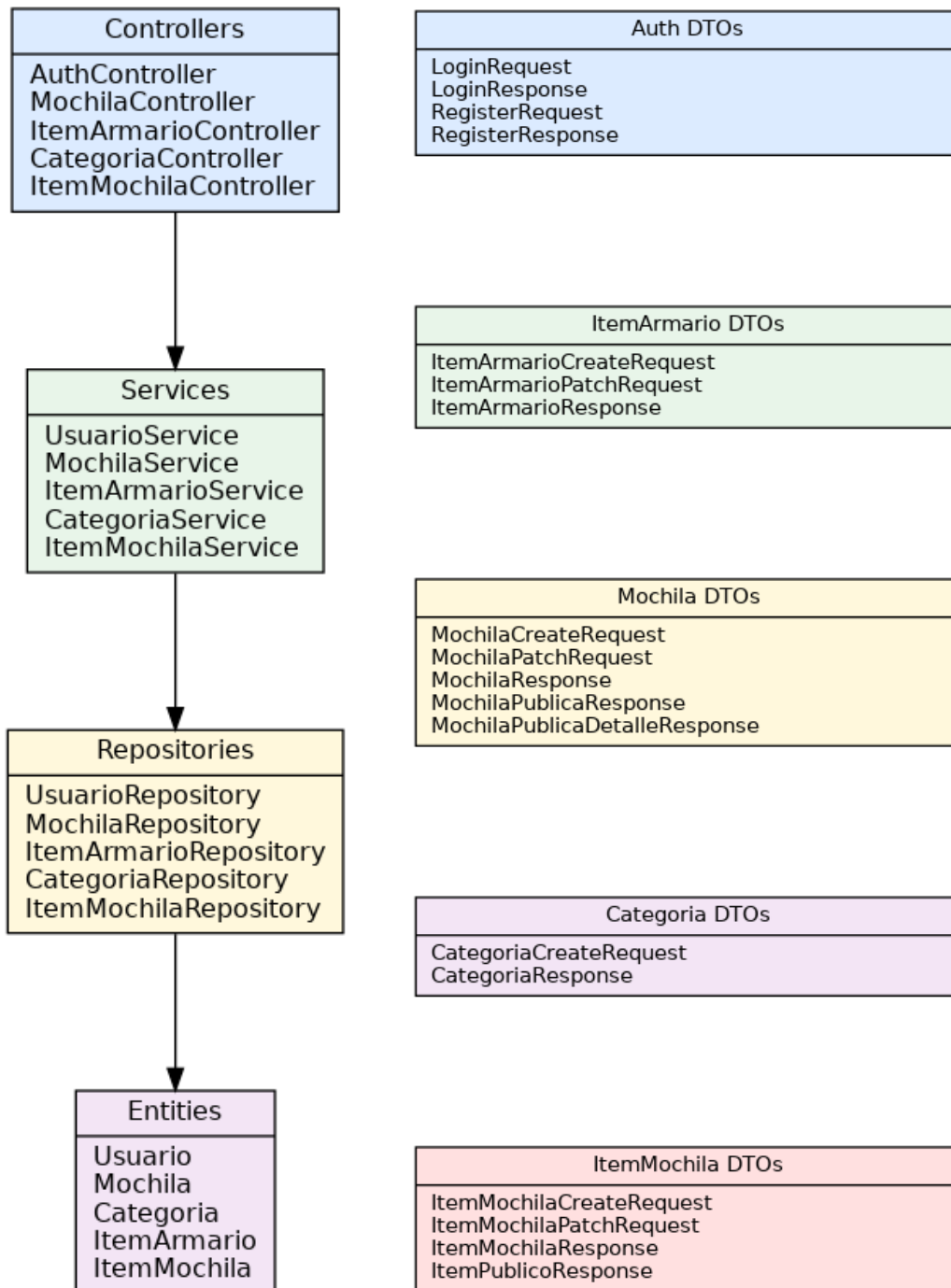
Items_armario		
Columna	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
nombre	VARCHAR	NOT NULL
peso	INT	en gramos
descripcion	VARCHAR	nullable
enlace	VARCHAR	nullable
usuario_id	BIGINT	FK → usuarios.id

Items_mochila		
Columna	Tipo	Restricciones
id	BIGINT	PK, AUTO_INCREMENT
cantidad	INT	NOT NULL
mochila_id	BIGINT	FK → mochilas.id
categoria_id	BIGINT	FK → categorias.id
item_armario_id	BIGINT	FK → items_armario.id

Relaciones

- USUARIOS 1:N MOCHILAS — Un usuario puede tener múltiples mochilas, pero cada mochila pertenece a un único usuario.
- USUARIOS 1:N ITEMS_ARMARIO — Un usuario puede tener múltiples objetos en su armario personal, pero cada objeto pertenece a un único usuario.
- MOCHILAS 1:N CATEGORIAS — Una mochila puede contener múltiples categorías, pero cada categoría pertenece a una única mochila.
- MOCHILAS 1:N ITEMS_MOCHILA — Una mochila puede contener múltiples items, pero cada registro de item está asociado a una única mochila.
- CATEGORIAS 1:N ITEMS_MOCHILA — Una categoría puede agrupar múltiples items, pero cada item pertenece a una única categoría dentro de la mochila.
- ITEMS_ARMARIO 1:N ITEMS_MOCHILA — Un objeto del armario puede reutilizarse en múltiples mochilas, pero cada registro de item_mochila referencia a un único objeto del armario.

Diagramas Spring



Arquitectura del backend

El backend de Ligerito se ha desarrollado utilizando Spring Boot siguiendo una arquitectura por capas, separando responsabilidades entre controladores, servicios, persistencia y entidades.

La estructura principal se divide en:

Controllers

Gestionan las peticiones HTTP recibidas desde el frontend React y exponen la API REST de la aplicación.

Entre los principales controladores destacan:

- AuthController
- MochilaController
- ItemArmarioController
- CategoriaController
- ItemMochilaController

Services

Contienen la lógica de negocio de la aplicación, desacoplando las reglas funcionales de la capa HTTP.

Esta capa se encarga de operaciones como:

- autenticación de usuarios,
- cálculo de pesos,
- sincronización del armario,
- validaciones,
- y gestión de mochilas y categorías.

Repositories

Gestionan el acceso a base de datos utilizando Spring Data JPA.

Cada repositorio trabaja sobre una entidad concreta y permite realizar operaciones CRUD de forma sencilla sin necesidad de escribir SQL manual.

Entidades JPA

Representan el modelo de datos persistente de la aplicación y su relación con las tablas MySQL mediante anotaciones JPA.

Las entidades principales son:

- Usuario
- Mochila
- Categoria
- ItemArmario
- ItemMochila

La comunicación entre frontend y backend se realiza mediante una API REST basada en JSON.

Base de datos

La persistencia de datos se ha implementado utilizando MySQL como sistema de gestión de base de datos relacional.

El modelo de datos se diseñó buscando:

- reutilización de equipamiento
- coherencia entre mochilas
- escalabilidad
- eliminación automática de dependencias mediante relaciones en cascada.

Uno de los aspectos más importantes del diseño es la separación entre:

- los objetos globales del armario (items_armario),
- y los objetos utilizados dentro de cada mochila (items_mochila).

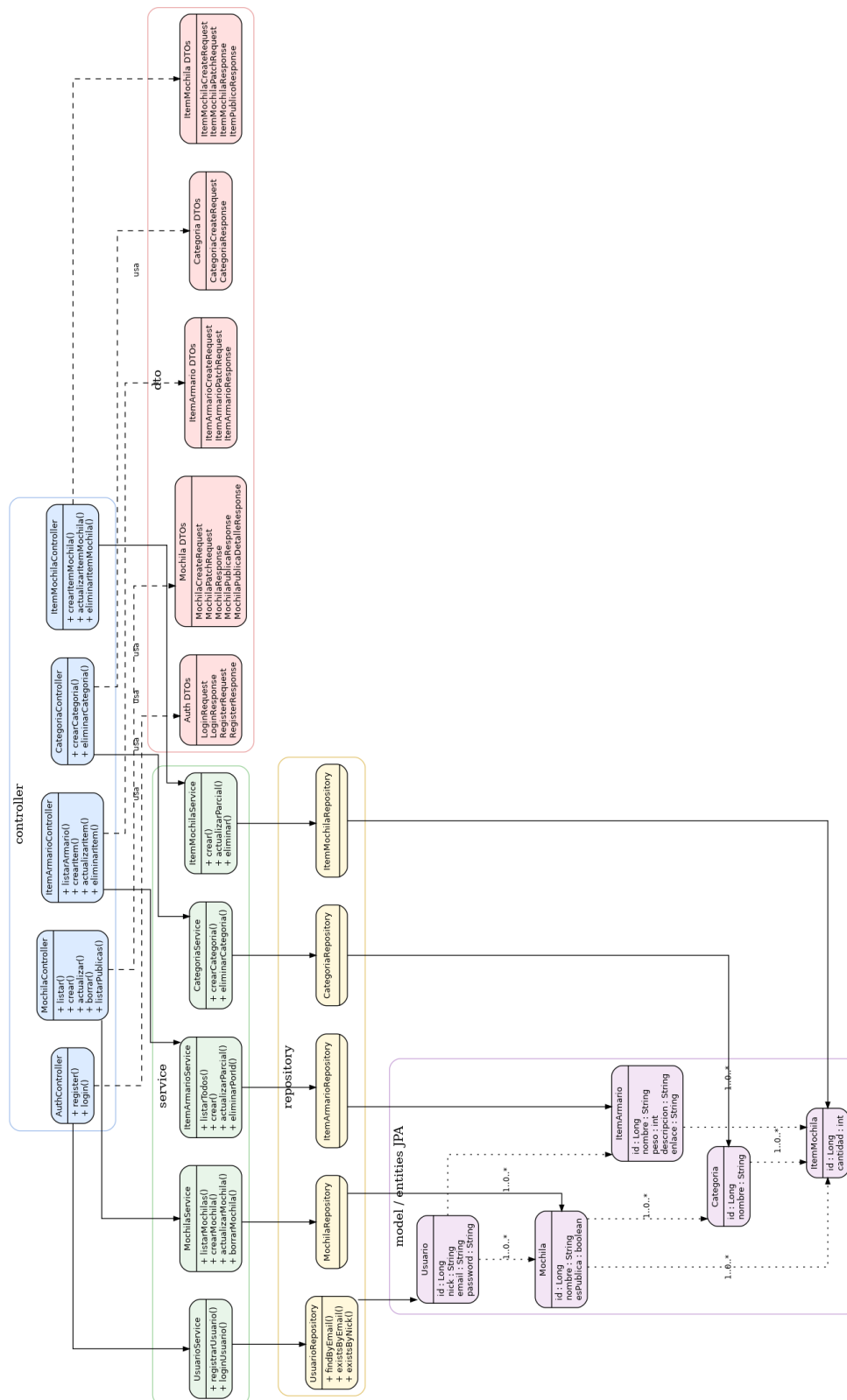
Gracias a esta estructura, un mismo objeto puede reutilizarse en múltiples mochilas sin duplicar información.

La persistencia se gestiona mediante Spring Data JPA e Hibernate, utilizando relaciones:

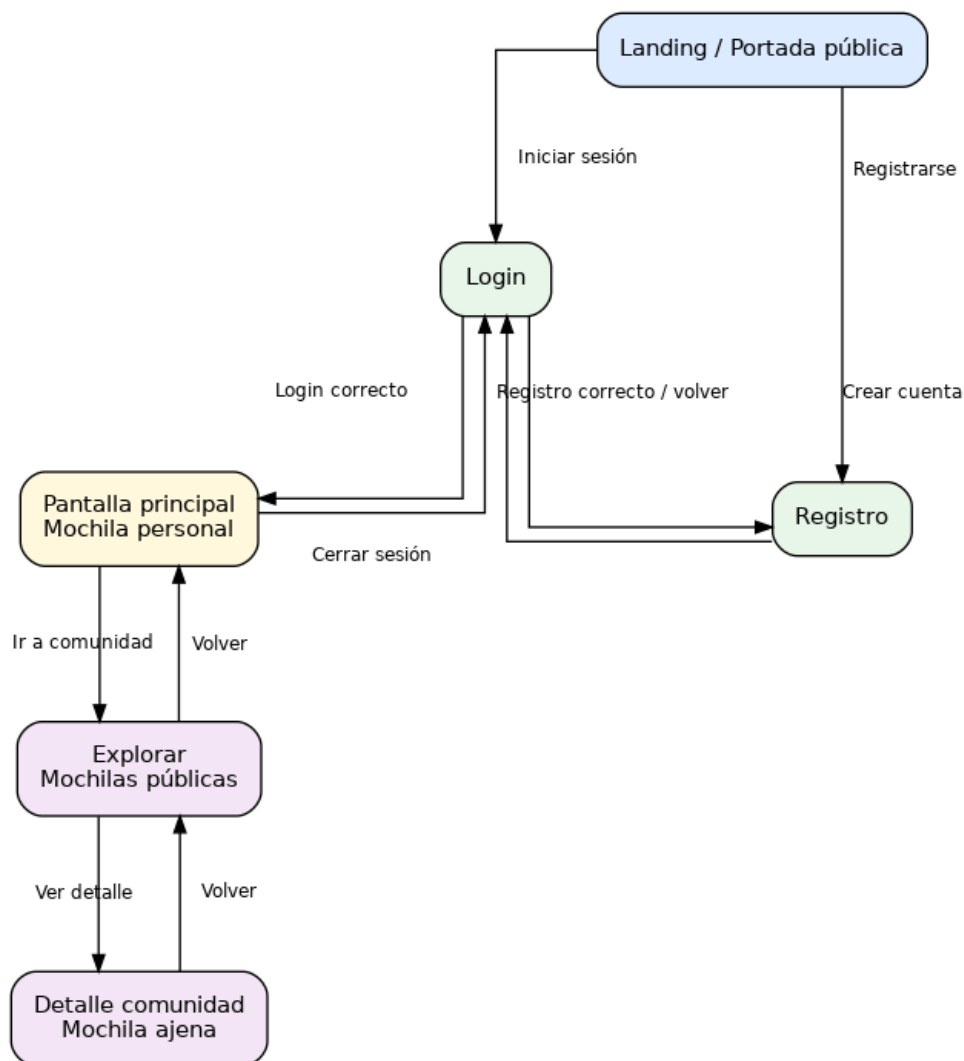
- @OneToMany
- @ManyToOne
- cascade
- orphanRemoval

Diagrama de clases completo.

[Ver diagrama en pantalla completa](#)



Mapa navegación entre pantallas.

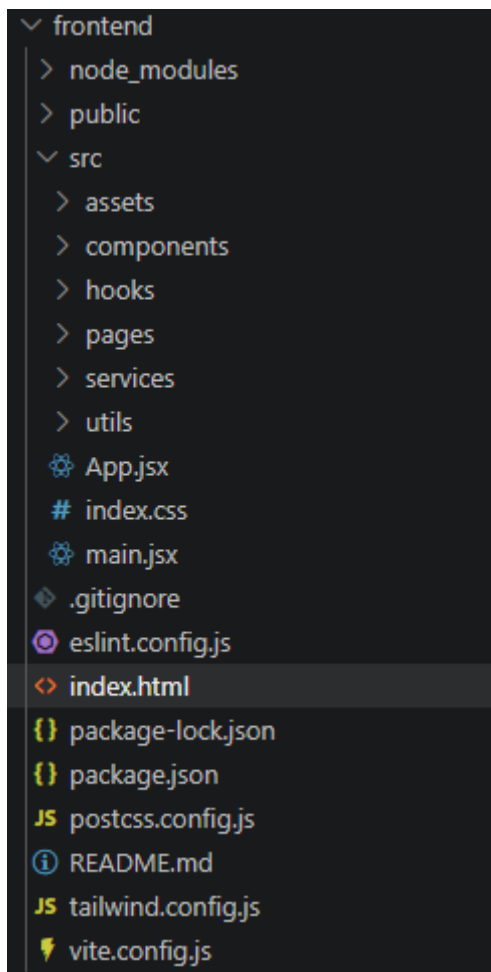


Estructura del frontend

El frontend de Ligerito se ha desarrollado utilizando React junto con Vite como herramienta de construcción del proyecto y Tailwind CSS para el diseño visual y la maquetación responsive.

La aplicación se organiza siguiendo una estructura modular basada en responsabilidades, separando componentes reutilizables, pantallas, hooks personalizados y servicios de comunicación con el backend.

La estructura principal del proyecto es la siguiente:



Carpeta public

La carpeta public contiene recursos estáticos utilizados por la aplicación, principalmente imágenes empleadas en la landing page y en diferentes vistas del sistema.

Entre estos recursos se incluyen capturas reales de la aplicación utilizadas para mostrar funcionalidades como:

- gestión del armario,
- distribución del peso,
- mochilas públicas,
- y visualización de categorías.

Carpeta src

La carpeta `src` contiene todo el código fuente del frontend React.

Components

La carpeta *components* almacena componentes reutilizables utilizados en distintas pantallas de la aplicación.

Algunos de los componentes principales son:

- **Sidebar.jsx:** navegación lateral entre secciones.
- **Header.jsx:** cabecera superior de la aplicación.
- **ListaCategorias.jsx:** representación visual de las categorías de una mochila.
- **Filaltem.jsx:** componente individual para representar un item dentro de una categoría.
- **ResumenPesos.jsx:** cálculo y visualización del peso total y por categorías.
- **FormularioNuevoItem.jsx:** formulario de creación de items.
- **ModalEnlace.jsx:** ventana modal para gestionar enlaces asociados a productos.
- **MochilaPDFView.jsx:** componente utilizado para generar la vista exportable en PDF.

Esta separación permite reutilizar lógica visual y mantener el código organizado y escalable.

Pages

La carpeta *pages* contiene las pantallas principales de la aplicación, asociadas a las distintas vistas que puede navegar el usuario.

Entre ellas destacan:

- **Landing.jsx:** página pública inicial del proyecto.
- **Login.jsx:** autenticación de usuarios.
- **Registro.jsx:** creación de nuevas cuentas.
- **Explorar.jsx:** visualización de mochilas públicas compartidas por otros usuarios.
- **DetalleComunidad.jsx:** vista detallada de una mochila pública.
- **Configuracion.jsx:** configuración y preferencias del usuario.

Cada página se compone de distintos componentes reutilizables para construir la interfaz completa.

Hooks personalizados

La carpeta *hooks* contiene hooks personalizados de React utilizados para encapsular lógica de estado y reutilizar comportamiento entre componentes.

Los principales hooks implementados son:

- useMochilas.js
- useMochilasBackend.js
- useArmario.js

Estos hooks gestionan operaciones relacionadas con:

- estado global de mochilas,
- sincronización con backend,
- gestión del armario,
- y actualización reactiva de datos.

El uso de hooks personalizados permite desacoplar la lógica de negocio de los componentes visuales.

Services

La carpeta *services* contiene la capa de comunicación con el backend Spring Boot mediante peticiones HTTP utilizando fetch.

Cada archivo encapsula las operaciones REST de una entidad concreta:

- apiAuth.js: login y registro de usuarios.
- apiArmario.js: operaciones CRUD del armario.
- apiMochilas.js: gestión de mochilas.
- apiCategorias.js: gestión de categorías.
- apiItemMochila.js: gestión de items dentro de las mochilas.

Esta separación facilita el mantenimiento y evita mezclar llamadas HTTP con lógica visual.

Utils

La carpeta *utils* contiene funciones auxiliares reutilizables.

Destacan:

- `exportarMochilaPDF.jsx`
- `prepararMochilaPDF.js`

Estas utilidades permiten generar documentos PDF exportables con el contenido de las mochilas del usuario.

App.jsx

App.jsx actúa como componente raíz de la aplicación.

Desde este archivo se controla:

- la navegación principal,
- la sesión del usuario autenticado,
- el renderizado condicional de pantallas,
- y la carga inicial de datos desde backend.

main.jsx

main.jsx es el punto de entrada de React y se encarga de montar la aplicación sobre el DOM utilizando ReactDOM

index.css y Tailwind CSS

El diseño visual se realiza mediante Tailwind CSS, permitiendo construir una interfaz responsive basada en utilidades CSS reutilizables.

El archivo *index.css* contiene estilos globales y configuración base de la aplicación.

Arquitectura general

El frontend sigue una arquitectura modular basada en separación de responsabilidades:

Pages → Components → Hooks → Services → Backend Spring Boot

De esta forma:

- las páginas gestionan la navegación,
- los componentes representan la interfaz,

- los hooks encapsulan lógica de estado,
- y los servicios gestionan la comunicación HTTP con el backend.

Resto de carpetas y archivos presentes:

node_modules/

Contiene las dependencias instaladas del proyecto. No se modifica manualmente y se genera a partir de package.json.

assets/

Carpeta reservada para recursos internos del frontend, como imágenes o iconos importados desde React. En el estado actual, la mayoría de imágenes públicas se gestionan desde public/.

package.json

Define las dependencias del proyecto, los scripts de ejecución y la configuración básica de npm.

package-lock.json

Archivo generado automáticamente que fija las versiones exactas de las dependencias instaladas.

vite.config.js

Configuración de Vite, herramienta utilizada para levantar el servidor de desarrollo y construir la aplicación.

tailwind.config.js

Configuración de Tailwind CSS, donde se definen rutas de escaneo y posibles extensiones del tema visual.

postcss.config.js

Configuración necesaria para procesar Tailwind CSS mediante PostCSS.

eslint.config.js

Configuración de ESLint para detectar errores y mantener calidad de código.

index.html

Documento HTML base donde se monta la aplicación React.

README.md

Archivo de documentación general del proyecto frontend.

.gitignore

Indica qué archivos o carpetas no deben subirse al repositorio, como node_modules.

Tecnologías utilizadas.

- React y Tailwind CSS para la construcción de interfaces dinámicas y responsive.
- Vite como herramienta de desarrollo y compilación del frontend.
- Spring Boot para la construcción del backend y la API REST.
- Spring Data JPA e Hibernate para la persistencia y gestión de entidades.
- Spring Security y BCrypt para la autenticación y protección de contraseñas.
- MySQL como sistema de persistencia relacional.
- Docker Compose para la ejecución del entorno de desarrollo.
- Git y GitHub para el control de versiones y gestión del proyecto.
- Herramientas de inteligencia artificial como ChatGPT y Gemini, utilizadas como apoyo durante el desarrollo para consultas técnicas, resolución de problemas, generación de diagramas y asistencia en determinadas decisiones de implementación.
- Stitch de Google para el diseño y prototipado de interfaces, permitiendo construir una interfaz visual moderna y profesional en un tiempo reducido.

Conclusiones

El desarrollo de Ligerito me ha permitido construir una aplicación web completa integrando un frontend desarrollado con React y un backend REST construido con Spring Boot.

El diseño y la implementación se hizo buscando la modularidad y separando por capas, creando una diferencia entre la lógica de presentación, la lógica de negocio y la persistencia de datos. Esto me permite mantener el código organizado, ser reutilizable y fácilmente escalable.

Uno de los aspectos más relevantes del proyecto ha sido el modelado del sistema de reutilización de equipamiento mediante la separación entre los objetos del armario y los items incluidos en las mochilas. Gracias a esta decisión de diseño, un mismo item puede reutilizarse en múltiples mochilas sin duplicar información, manteniendo coherencia en los datos y simplificando la gestión del inventario personal.

Además, se han implementado funcionalidades avanzadas como:

- autenticación de usuarios,
- mochilas públicas compartibles,
- organización por categorías,
- cálculo automático de pesos,

- exportación a PDF,
- y sincronización entre frontend y backend mediante peticiones HTTP.

Desde el punto de vista del aprendizaje, el proyecto ha permitido comprender de forma práctica conceptos fundamentales del desarrollo full stack, como el diseño de APIs REST, la gestión de estado en React, el modelado relacional de bases de datos, la separación por capas en Spring Boot y la comunicación cliente-servidor.

Como posibles mejoras futuras, el proyecto podría ampliarse incorporando funcionalidades como:

- generación automática de mochilas mediante inteligencia artificial.
- sistema de recomendaciones de equipamiento.
- Incorporar una checkbox en cada item para controlar la preparación de cada mochila.

El proyecto ha permitido desarrollar una aplicación funcional y extensible, aplicando tecnologías y patrones utilizados en entornos reales de desarrollo web moderno.

Como conclusión personal, el proyecto me ha permitido unir muchos de los conocimientos trabajados por separado durante el ciclo formativo y aplicarlos en una aplicación completa y funcional, obteniendo un resultado tangible y cercano a un entorno profesional real.